

Android Application Penetration Testing

Academic Android application penetration testing of an Indonesian service application. The application identity is withheld in this public portfolio.

The documented assessment was conducted as an academic group project in an authorized, non-destructive dummy/testing context.

Overview

This project combined Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) to evaluate selected security properties of the dummy Android application.

Documented assessment context:

- Application: Anonymized Android service application (dummy/testing assessment context)
- Version: 4.0.19
- versionCode: 203
- Testing context: authorized, non-destructive
- Environment: dummy/testing environment and account
- Classification: OWASP/MASVS-related mapping and CVSS v3.1

The final revised assessment documented 3 High, 5 Medium, and 1 Low CVSS-scored findings, plus 4 positive/informational security observations. The highest documented severity was CVSS v3.1 8.4 (High).

My Role

Role: Technical Testing Coordinator

My contribution focused primarily on the technical security-testing portion of the academic group project, including static and dynamic application analysis, vulnerability validation, technical evidence collection, and security finding assessment.

The final report and documentation were produced as a group project; this repository does not claim sole authorship of all project documentation.

Methodology

Static Analysis

- APK decompilation and code inspection
- AndroidManifest.xml review
- Resource and configuration review
- Static keyword searches
- OAuth-flow review
- Network-security configuration review

Primary tools and artifacts:

- JADX / JADX-GUI
- AndroidManifest.xml
- strings.xml
- network_security_config.xml

Dynamic Analysis

- HTTP(S) traffic inspection
- Controlled request replay
- Authentication and session validation
- OAuth request review
- Runtime log inspection

- Local application-storage inspection

Primary tools:

- Burp Suite Community
- LDPlayer Android Emulator
- ADB / logcat
- JWT debugger

Sensitive tokens, cookies, identifiers, account data, and other private values were removed from the public portfolio.

Assessment Results

Key risk areas included:

- Sensitive authentication information exposed through runtime logging
- Authentication/session invalidation weaknesses
- Access to private data without valid authentication
- OAuth / PKCE-related weaknesses
- Insecure network configuration
- Client-side API/Firebase configuration exposure

See Sanitized Findings Summary for the public finding overview.

Positive Security Observations

The assessment also documented controls that behaved correctly during testing, including:

- Encrypted local-storage indicators
- Rejection of modified and unsigned JWTs
- No plaintext application user credential discovered in tested local-storage directories
- HTTPS observed during tested runtime flows

See Positive Security Controls.

Remediation

Highest-priority recommendations included:

1. Prevent sensitive authentication data from being written to production logs
2. Implement server-side session/token revocation
3. Enforce authentication and authorization for private resources
4. Implement PKCE S256 for native OAuth flows

See Remediation Priorities.

Repository Structure

- docs/ - sanitized assessment summaries
- evidence/ - public evidence-handling policy
- report/ - recruiter-friendly public portfolio report
- DISCLAIMER.md - scope, ethics, and disclosure limitations

Public Portfolio Report

A recruiter-friendly public summary is available here:

[Download](#) / [view the portfolio report](#)

Important Scope Note

This repository represents an academic assessment conducted in an authorized dummy/testing context.

It must not be interpreted as a statement about the current security posture of the production application or infrastructure.

Detailed reproduction steps, sensitive identifiers, authentication material, exact private request data, and other potentially harmful information have intentionally been omitted from the public portfolio.

Skills Demonstrated

- Mobile Application Security Testing
- Static Application Security Testing (SAST)
- Dynamic Application Security Testing (DAST)
- Android Application Analysis
- HTTP Traffic Analysis
- Authentication and Session Testing
- Vulnerability Validation
- Security Evidence Collection
- OWASP / MASVS Mapping
- CVSS v3.1 Assessment
- Security Remediation Analysis

Scope and Methodology

Assessment Scope

This project assessed a anonymized Android service application in a dummy/testing assessment context in an authorized, non-destructive testing context.

Documented target:

- Android application: Anonymized Android service application
- Version: 4.0.19
- versionCode: 203
- Testing environment: dummy/testing environment and account

The assessment combined static and dynamic analysis.

Static Application Security Testing

Static analysis reviewed:

- Decompiled APK content
- AndroidManifest.xml
- Application resources
- Network-security configuration
- Static indicators of application endpoints and authentication flows
- OAuth-related implementation patterns
- Local-storage indicators

Tools and artifacts included:

- JADX / JADX-GUI
- AndroidManifest.xml
- strings.xml
- network_security_config.xml

Dynamic Application Security Testing

Dynamic analysis evaluated selected runtime behaviors through:

- HTTP(S) traffic capture and inspection
- Controlled request replay
- Authentication/session validation
- OAuth request inspection
- Runtime log review
- Local application-storage inspection

Tools included:

- Burp Suite Community
- LDPlayer Android Emulator
- ADB shell / logcat
- JWT debugger

Data Handling

Sensitive information was removed from public assessment evidence.

Public portfolio material must not expose:

- Passwords
- Access tokens
- Refresh tokens
- Session cookies
- Authorization headers
- Personal email addresses
- User identifiers
- Private report identifiers
- Other sensitive or account-specific information

Assessment Boundaries

The project does not establish complete security coverage of the application.

Areas documented as requiring future testing included:

- SSL/TLS certificate-pinning resilience
- Deep-link and intent handling
- WebView security, including XSS, file access, and mixed content

These areas were not converted into confirmed findings without sufficient runtime evidence.

Sanitized Findings Summary

The table below summarizes CVSS-scored findings from the final revised assessment.

Detailed exploitation instructions, raw requests, authentication material, private identifiers, and sensitive technical evidence are intentionally excluded from this public portfolio.

ID	Finding	Source	CVSS v3.1	Severity	Status
D01	Sensitive authentication data exposed through runtime logging	Dynamic	8.4	High	Confirmed
D02	Private report comments accessible without valid authentication	Dynamic	7.5	High	Confirmed
D03	Session/access token remains valid after logout	Dynamic	8.1	High	Confirmed
F01	Insecure network configuration permitting cleartext traffic	Static	5.4	Medium	Static confirmed; runtime impact not confirmed
F02	OAuth custom URI scheme with no static evidence of PKCE	Static	6.8	Medium	Potential; runtime behavior assessed separately
F03	Potential OAuth authorization-code logging	Static	6.1	Medium	Potential
D04	OAuth authorization request without PKCE parameter	Dynamic	6.8	Medium	Confirmed on tested request
D05	Long-lived JWT and readable claims	Dynamic	5.3	Medium	Supporting evidence
F05	Firebase / Google API configuration exposure	Static	3.7	Low	Potential

Main Risk Themes

Authentication Data Exposure

Runtime logging exposed sensitive authentication-related data during testing, creating a potential credential and token exposure risk.

Session Management

Testing showed that an existing authenticated token could remain usable after logout, indicating insufficient session revocation behavior in the tested environment.

Access Control

A private-data access scenario was validated without valid authentication during the controlled assessment.

OAuth Security

Static and dynamic observations identified weaknesses related to PKCE usage and sensitive OAuth-related logging.

Configuration Security

Static analysis identified network and API configuration issues that warranted additional hardening.

Interpretation

The results represent the behavior of the specific dummy/testing environment and application version assessed during the project.

They should not be interpreted as evidence that the same conditions currently exist in any production environment.

Positive Security Controls

Security assessment should document not only weaknesses but also controls that behaved correctly during testing.

The final revised assessment recorded the following positive or informational observations.

ID	Observation	Result
PF01	Local data-encryption indicators	EncryptedSharedPreferences and restrictive backup configuration were observed
D06	JWT tampering / unsigned token validation	Modified and alg:none tokens were rejected
D07	Local credential-storage review	No plaintext application user token or password was found in the tested application directories
D08	Runtime transport validation	Tested runtime flows used HTTPS; sensitive cleartext HTTP traffic was not confirmed

Why These Matter

These observations prevent the assessment from presenting the application as universally insecure.

They also demonstrate an evidence-based testing approach: a behavior is only classified as vulnerable when supporting evidence exists.

For example, although static network configuration required improvement, the tested runtime flow did not provide evidence of sensitive cleartext HTTP traffic.

Similarly, local-storage testing did not identify plaintext application authentication credentials in the directories examined.

Remediation Priorities

The final assessment prioritized remediation based on security impact and estimated implementation effort.

Priorty	Recommendation	Related Findings	Effort	Impact
1	Disable sensitive BODY-level HTTP logging in production and restrict verbose logging to debug builds	D01 / F03	Low	High
2	Implement server-side session/token revocation during logout	D03 / D05	Medium	High
3	Require authentication and authorization checks for private report resources	D02	Medium	High
4	Implement PKCE S256 for native OAuth flows	F02 / D04	Medium	Medium
5	Remove unnecessary cleartext-traffic permissions and validate runtime transport security	F01 / D08	Low-Medium	Medium
6	Restrict exposed Google/Firebase API configuration and review Firebase security rules	F05	Low-Medium	Medium

Remediation Philosophy

The recommendations prioritize weaknesses that could expose authentication material or enable unauthorized access before lower-impact configuration issues.

The assessment also distinguishes between:

- confirmed runtime vulnerabilities,
- static security weaknesses,
- supporting evidence,
- positive security controls,
- and future-testing areas.

This prevents potential weaknesses from being presented as confirmed vulnerabilities without sufficient evidence.

Disclaimer

This repository is an academic cybersecurity portfolio artifact.

The assessment documented here was performed in an authorized, non-destructive dummy/testing context as part of a university project.

This repository does not represent an official security assessment performed on behalf of the application owner, service operator, or any government organization.

The findings represent observations from the specific application version, environment, and testing conditions documented during the academic project.

They must not be interpreted as statements about the current security posture of any production system.

Sensitive technical details, private identifiers, authentication material, exact exploitation sequences, and other information that could enable misuse have intentionally been removed from the public version.

This repository is intended to demonstrate cybersecurity assessment methodology, technical analysis, evidence handling, risk classification, and remediation reasoning.

Security testing should only be performed on systems for which explicit authorization has been granted.

Evidence Handling

Only selected, sanitized evidence should be included in this public portfolio.

The original academic assessment contained technical screenshots and testing artifacts. Public evidence is intentionally limited to avoid exposing:

- authentication credentials,
- access or refresh tokens,
- cookies,
- authorization headers,
- personal information,
- user identifiers,
- private object/report identifiers,
- sensitive request or response bodies,
- exact exploitation sequences,
- reusable authentication material.

Recommended Public Evidence

Safe examples include:

1. Sanitized JADX / manifest-review screenshot
2. Sanitized Burp Suite interface showing the testing workflow without sensitive request data
3. Sanitized ADB / local-storage evidence demonstrating encrypted-storage indicators
4. Sanitized JWT-validation evidence showing rejected tampered tokens

Screenshots should support the methodology without publishing unnecessary exploitation details.

Redaction Standard

Before publishing any screenshot:

- Remove or blur tokens and cookies
- Remove passwords and account information
- Remove email addresses and personal identifiers
- Remove unique report or object identifiers
- Remove private hostnames if applicable
- Remove authorization headers
- Remove sensitive request/response bodies
- Verify the exported image again before publishing

When in doubt, omit the screenshot.